B+树介绍
B+树的基本概念：
B+树的基本操作：
B+树的查找
B+树的插入
B+树的删除

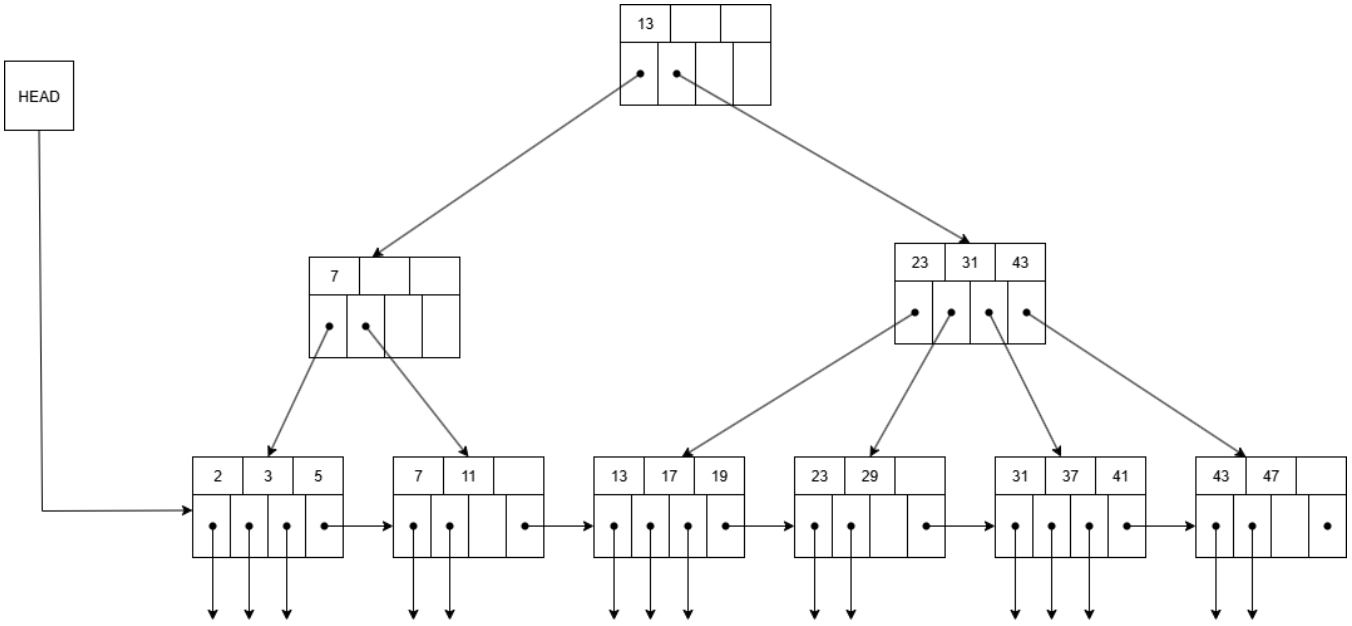
B+树介绍

相比我们熟悉的二叉查找树，B+树是一种**多路平衡查找树**，它专门为**磁盘等外存存储优化设计**，能够高效支持大规模数据的**查找、插入和删除**操作，尤其擅长**范围查询。**且哟要明确B+树的一个重要的特点，节点内部的关键字是有序排列的。

B+树的基本概念：

什么是B+树的阶?

B+树中一个节点的子节点数量的最大值，用m表示，如下图所示，节点所能容纳的子节点的数量的最大值为4，所以m = 4



所有节点中，【23，31，,43】节点拥有的子节点的数量最多，共有4个子节点，可以定义上述B+树为4阶B+树。

B+树基础概念：

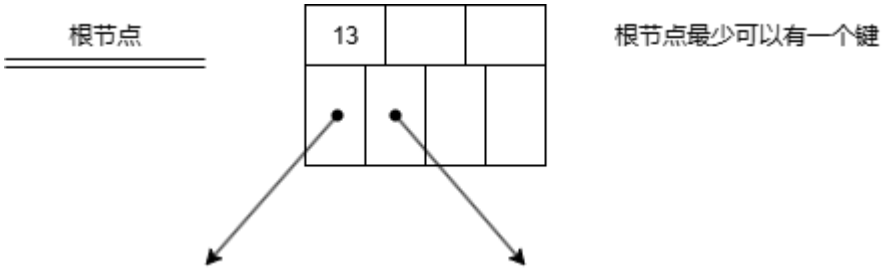
B+树是一种平衡的多叉树，通常我们说一颗m阶的B+树，它要满足如下条件：

- 非叶子节点最多有m个子节点
- 对于非叶子节点（根节点除外），最每个节点最少要有[m / 2]个子节点
- 如果根节点不是叶子节点，则至少包含两个子节点
- 具有k个子节点的非叶子节点，具有k-1个key
- 所有叶子节点距根节点的高度一致，且通过指针链接起来

在B+树中，只有叶子节点才存储实际的数据，非叶节点存储的是用于定位叶子节点的索引键。我们可以把B+树理解为一种多级索引的结构。

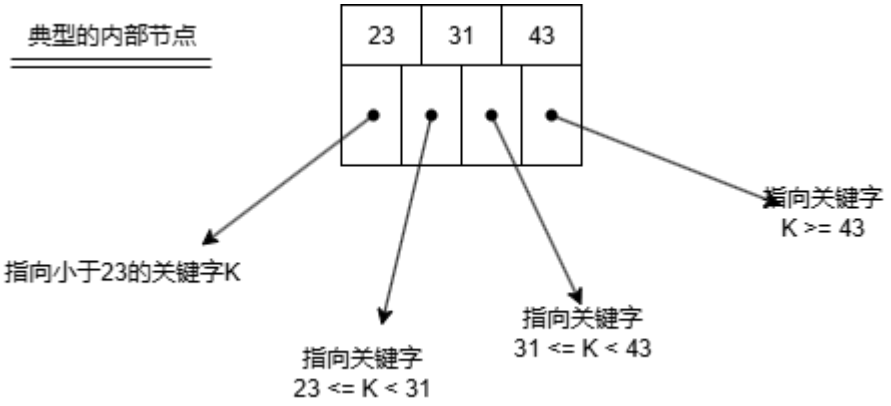
什么是根节点?

节点【30】即为根节点，特征：根节点拥有的子节点数量的上限和内部节点相同，如果根节点不是树中唯一节点的话，至少有俩个子节点（不然就变成单支了）。在m阶B+树中（根节点非树中唯一节点），那么有关系式2<= M <=m，M为子节点数量；包含的元素数量 1<= K <=m-1,K为元素数量。



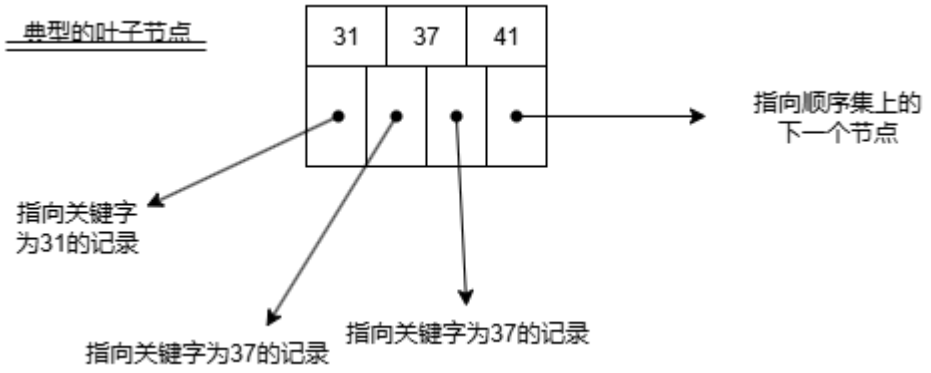
什么是内部节点?

节点【20】、节点【40, 50】都为内部节点，特征：内部节点是除叶子节点和根节点之外的所有节点，拥有父节点和子节点。假定m阶B+树的内部节点的子节点数量为M，则一定要符合（m/2）<= M <=m关系式，包含元素数量M-1；包含的元素数量 （m/2）-1<= K <=m-1,K为元素数量。m/2向上取整。



什么是叶子节点?

节点【10】、节点【20】、节点【30】等最后一层都为叶子节点，叶子节点对元素的数量有相同的限制，但是没有子节点，也没有指向子节点的指针。特征：在m阶B+树中叶子节点的元素符合（m/2）-1<= K <=m-1。



B+树的基本操作：

下面以一颗4阶的B+树为示例：

对于一颗m阶的B+树，对于非根的内部节点来说，最多有m个子节点，最少有【m / 2】（上取整）个子节点，关键字的个数 + 1 = 子节点的个数，所以关键字的个数最多有【m - 1】个，最少有【m / 2】 - 1个，而对于叶子节点，关键字个数和子节点的个数相等，所以对于叶子节点，关键字的个数和子节点的个数相等，都是m - 1,有一个子节点是指向下一个兄弟节点的。都是【（m - 1）/ 2】个关键字和子节点。

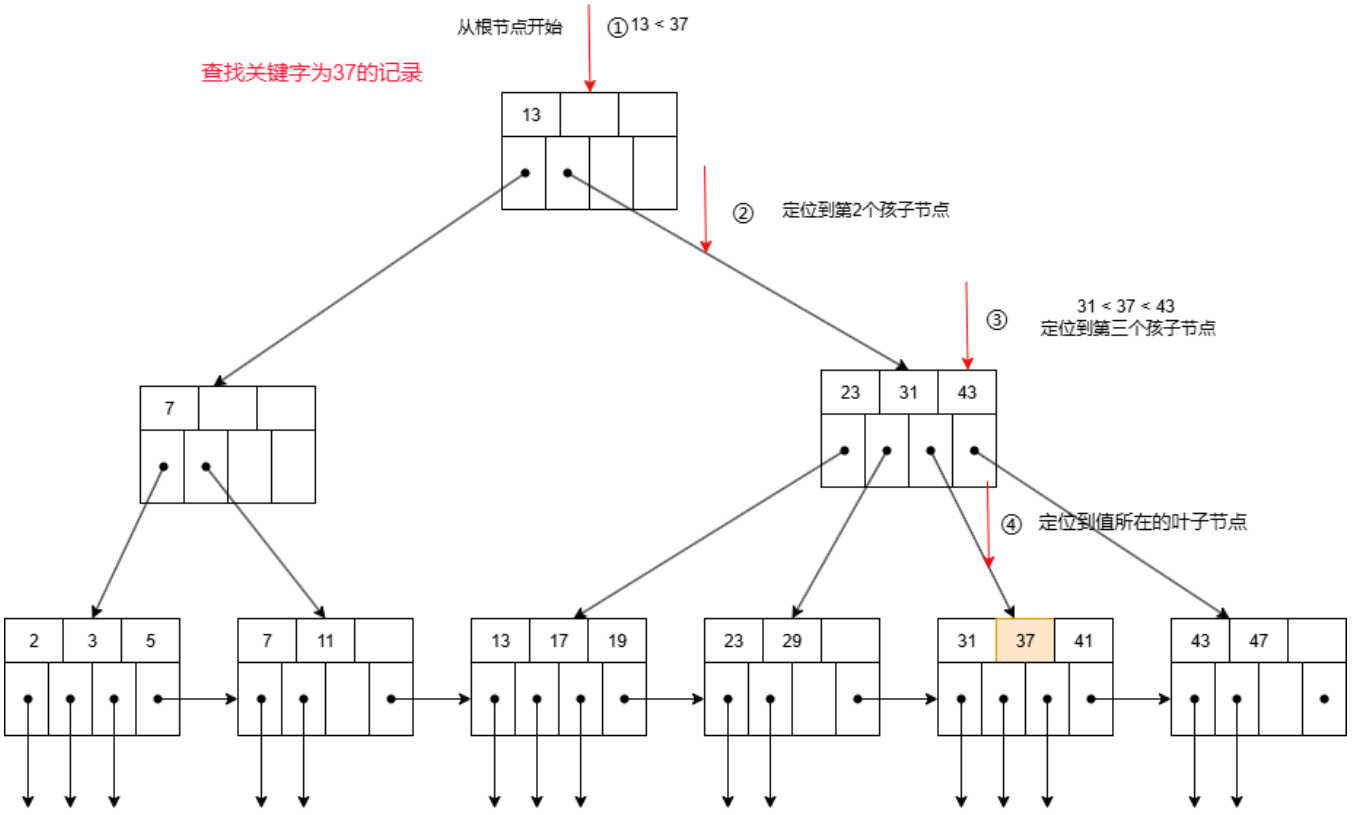
那么对于一颗4阶的B+树，非根的内部节点最少就有2个子节点，和1个关键字，最多有4个子节点和三个关键字；叶子结点最少就有2个关键字和2个子节点，最多有m - 1个子节点和m - 1个关键字；（注：这里没有把指向下一个兄弟的子节点算入在内）

B+树的查找

B+树通常可以进行两种查找运算：

1）随机查找：给定关键字key，在B+树中找到对应的记录。

- **查找关键字为37的记录**



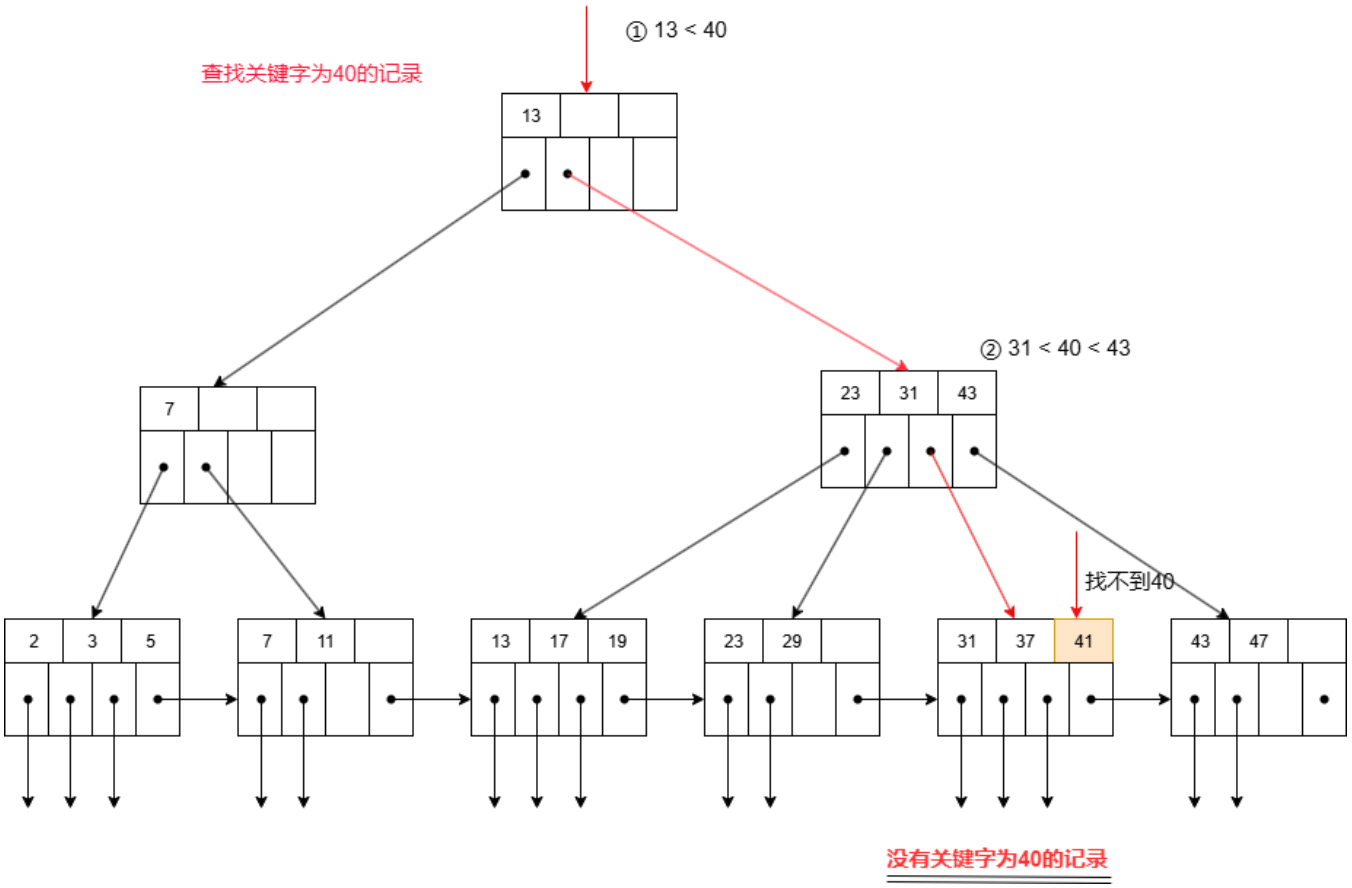
①首先从根节点开始，根节点有一个关键字13,13 < 37,所以定位到第二个孩子节点；

②通过第二个孩子节点找到其孩子节点；

③ 31 < 37 < 43，定位到37所在的叶子节点

④ 在叶子节点中找到37对应的记录；

- **查找关键字为40的记录**



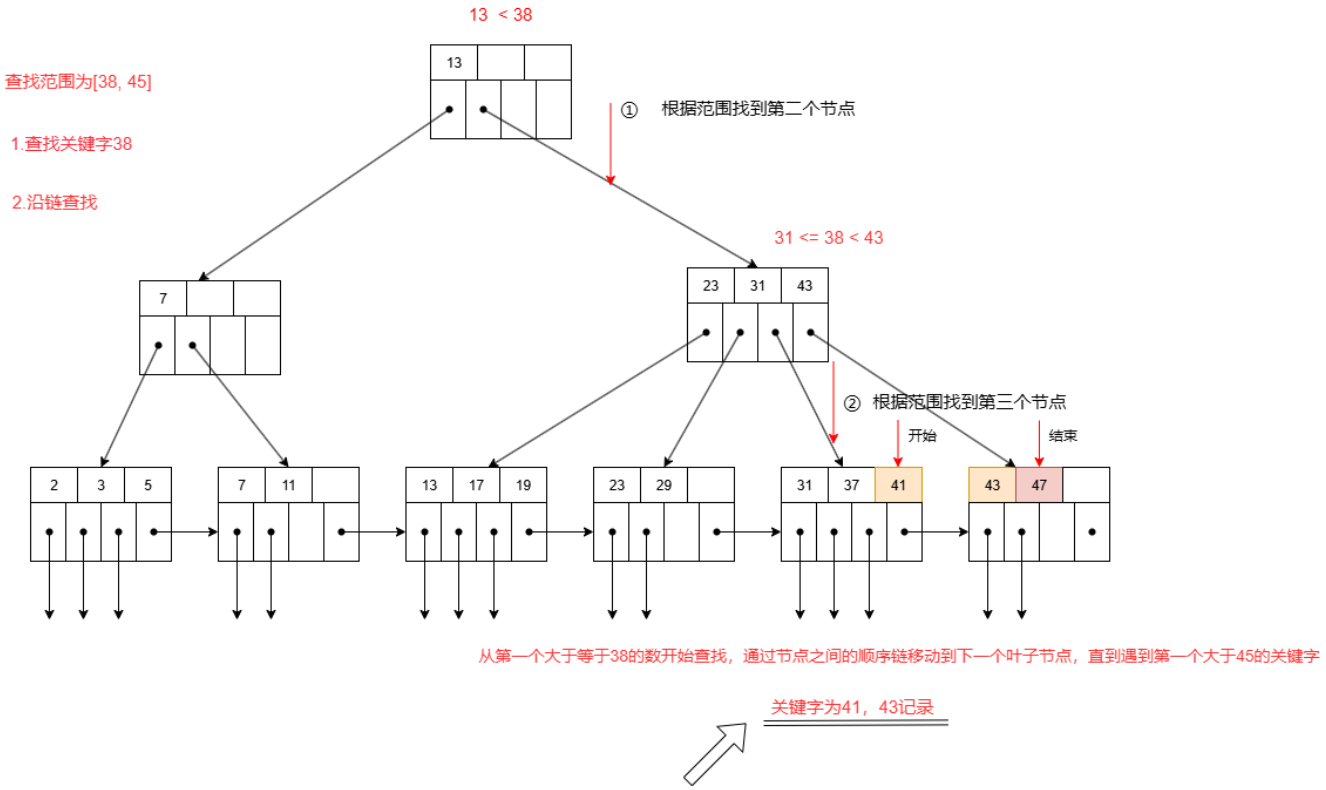
① 根据根节点中的关键字定位40所在的子节点

② 根据内部节点中的值可以判断31 < 40 < 43，找到关键字40应该在的叶子节点

③ 在叶子节点中查找关键字为40的记录。

没有查找到关键字为40的记录，可以断定底层的数据文件中并没有关键字为40的记录。

2）范围查找：给定上界和下界，找出在范围内的所有记录。



进行范围查找的时候，首先还是使用随机查找，找到下界所在的节点；在叶子节点上找到第一个大于等于下界的关键字，从该关键字开始沿着链进行遍历，把关键字对应的记录拿出来,直到找到第一个大于上界的关键字。

> B+树每一层都是有序的，都可以使用二分进行加速查找；在内部节点中可以使用upper_bound来快速定位，叶子节点中可以使用lower_bound进行定位。

B+树的插入

对于m阶的B+树，插入一个元素的时候，首先找到该关键字所在的叶子节点，判断该关键字在B+树的叶子结点中是否存在，如果存在，说明key对应的元素已经插入，则不需要插入新的元素，代表本次插入操作执行完毕。

对于m阶的B+树而言，假设要插入关键字`a`,首先根据索引键找到`a`所在的叶子结点`x`，然后在该叶子结点中合适的位置上插入该关键字`a`后执行下面逻辑：

1. 如果插入后该节点中关键字的个数小于等于m - 1,则此次插入操作逻辑结束。
2. 如果插入后该节点中的关键字的数量大于m - 1，则需要进行分裂操作；从老节点中拿出一半的元素到新节点中。
    1. 对于叶子节点：将节点一分为二，并把新节点的首个关键字插入其父节点中作为索引键。
    2. 对于内部节点，需要将中间键上移到父节点中，左右部分各自作为新的子节点。
    3. 如果分裂的是根节点，则创建新的根节点，并将中间键作为新根的唯一键，指向两个新子节点。
    4. 如果父节点关键字数量溢出，递归处理父节点。

3.重复上述操作，直到B+树达到平衡。

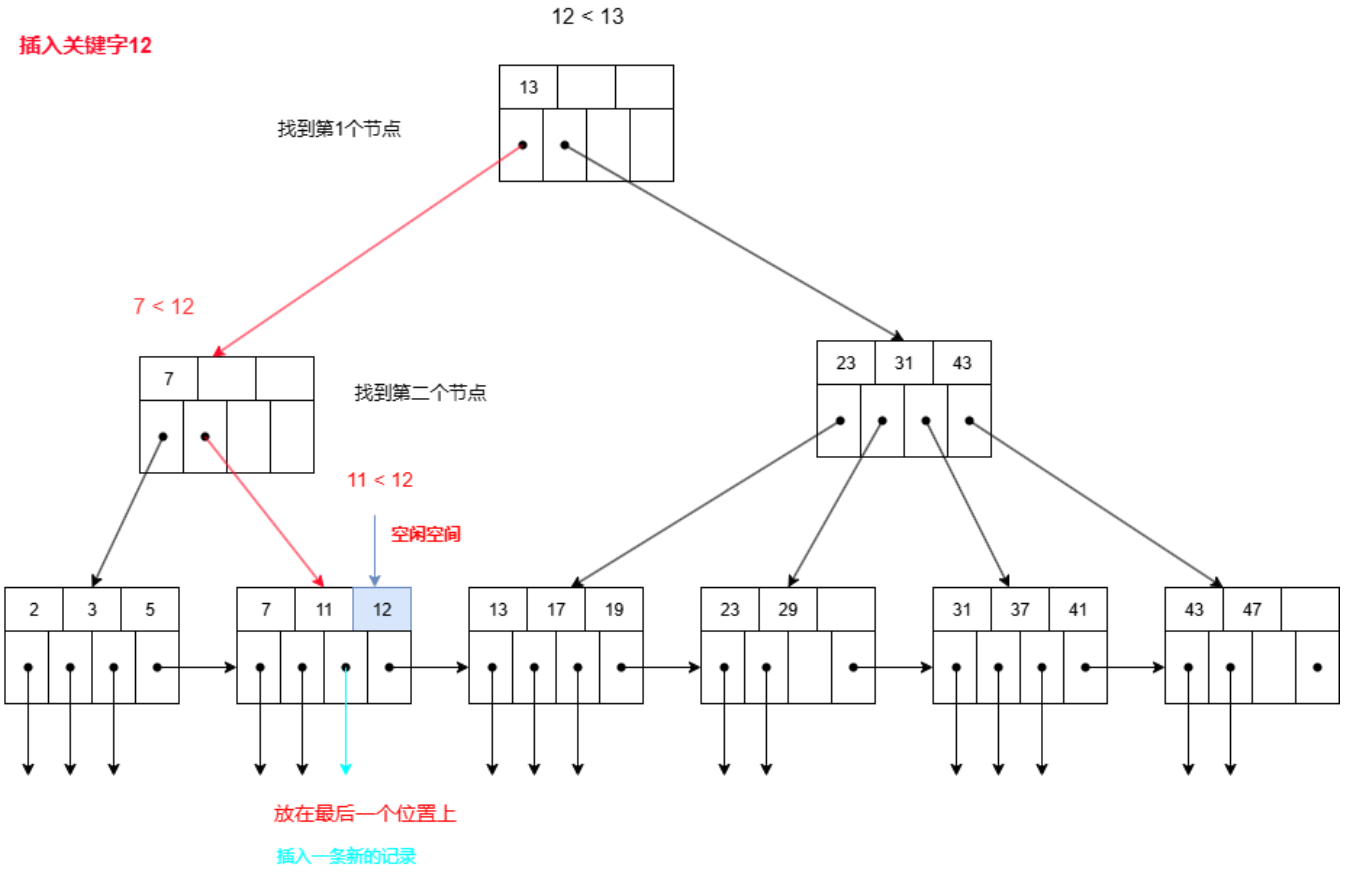
所以B+树的插入共有下面**三种情况**：

1. 叶节点上有空闲空间
2. 叶节点满，父亲节点（内部节点有空闲空间）
3. 内部节点满

1. 根节点有空闲空间
2. 根节点无空闲空间

(1）叶节点上有空闲空间

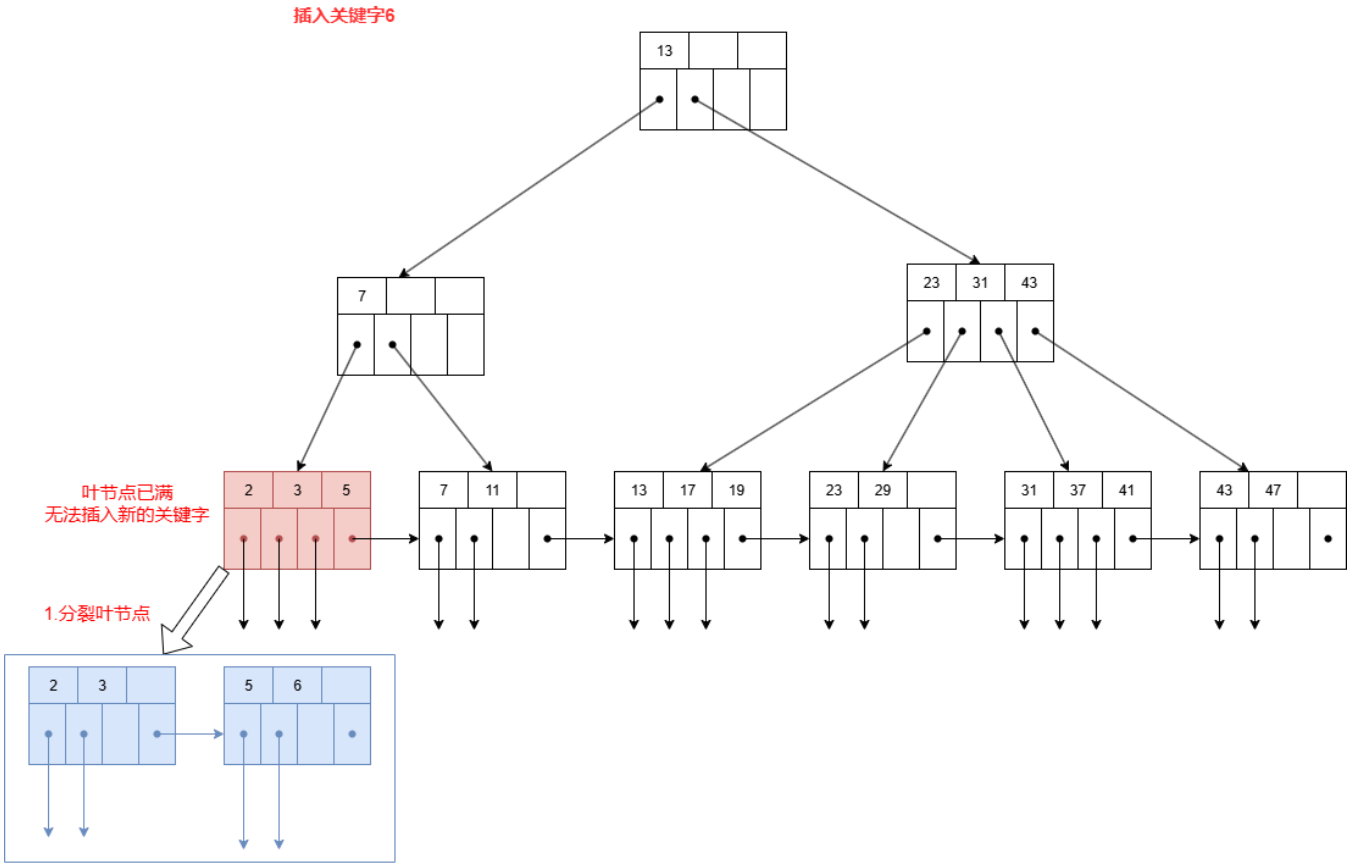
- **插入关键字12**



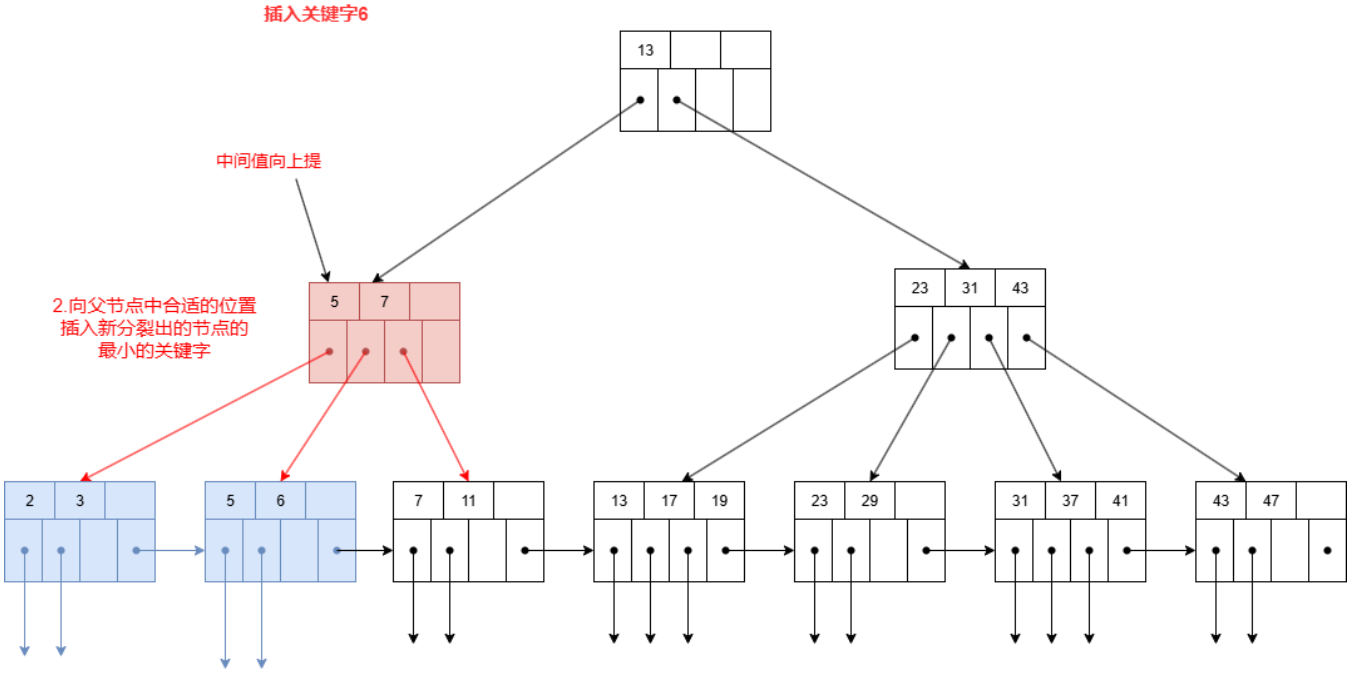
当叶子节点上存在空闲空间的时候，直接在适合的位置插入关键字。

(2）叶节点满，父亲节点（内部节点）有空闲

① 分裂叶子节点，取一半的值给新叶子节点



② 将中间值向上提，插入到父节点中合适的位置

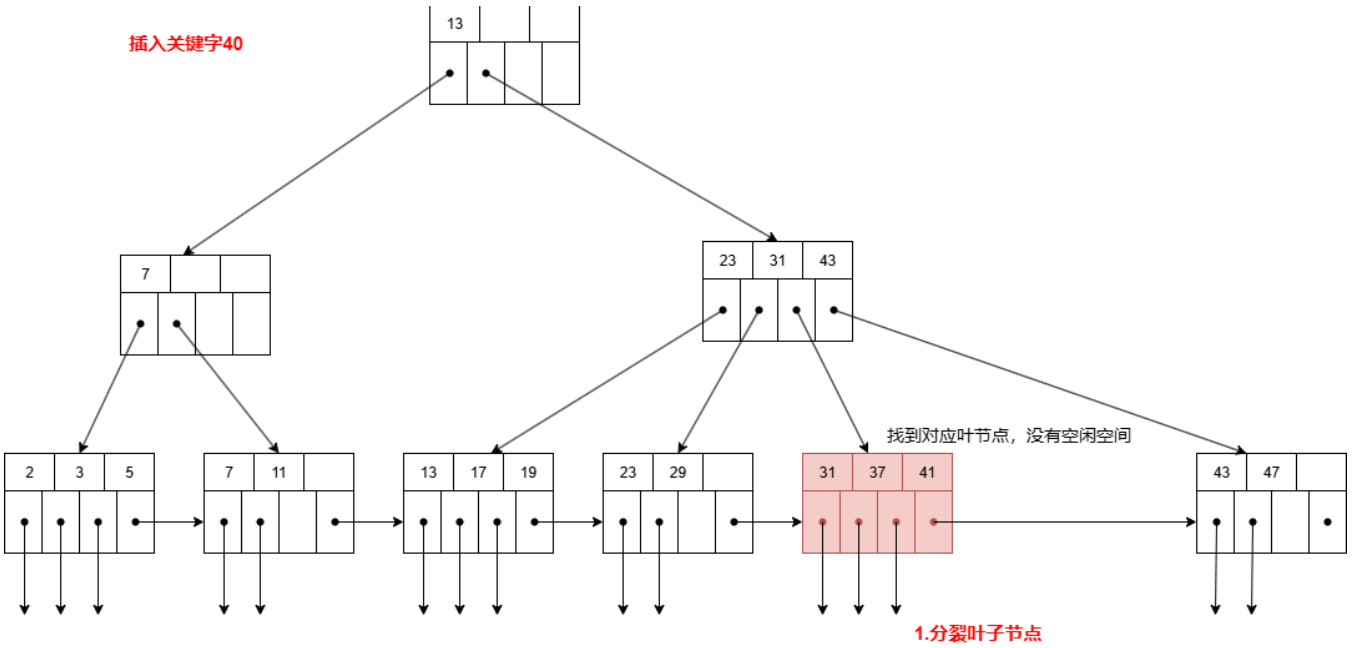


如果插入对应的叶节点之后，叶节点中关键字的数量大于最大个数，则需要将叶子节点进行分裂，并将中间值（新节点的最小值）提到老节点的父亲节点的合适的位置上，父亲节点有空闲位置，就直接插入到合适的位置上，如果没有空闲位置，对应操作继续往下看。

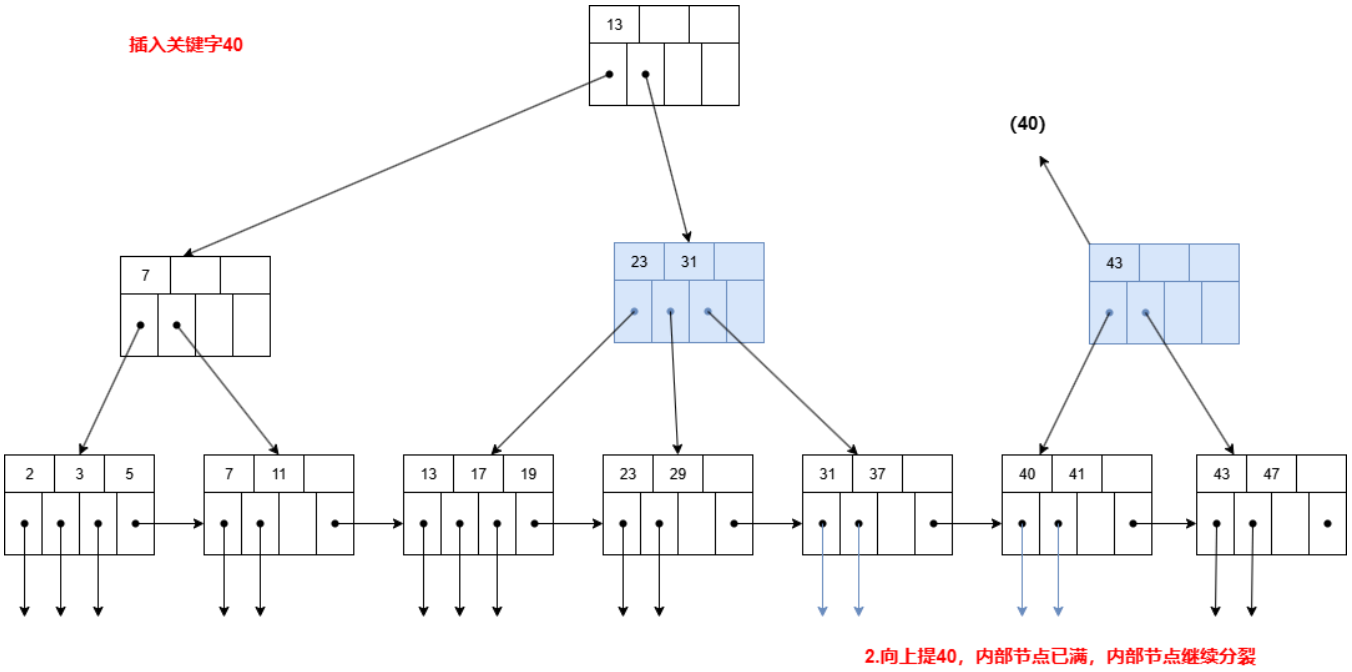
（3）内部节点满

1. **内部节点满，根节点有空闲空间**

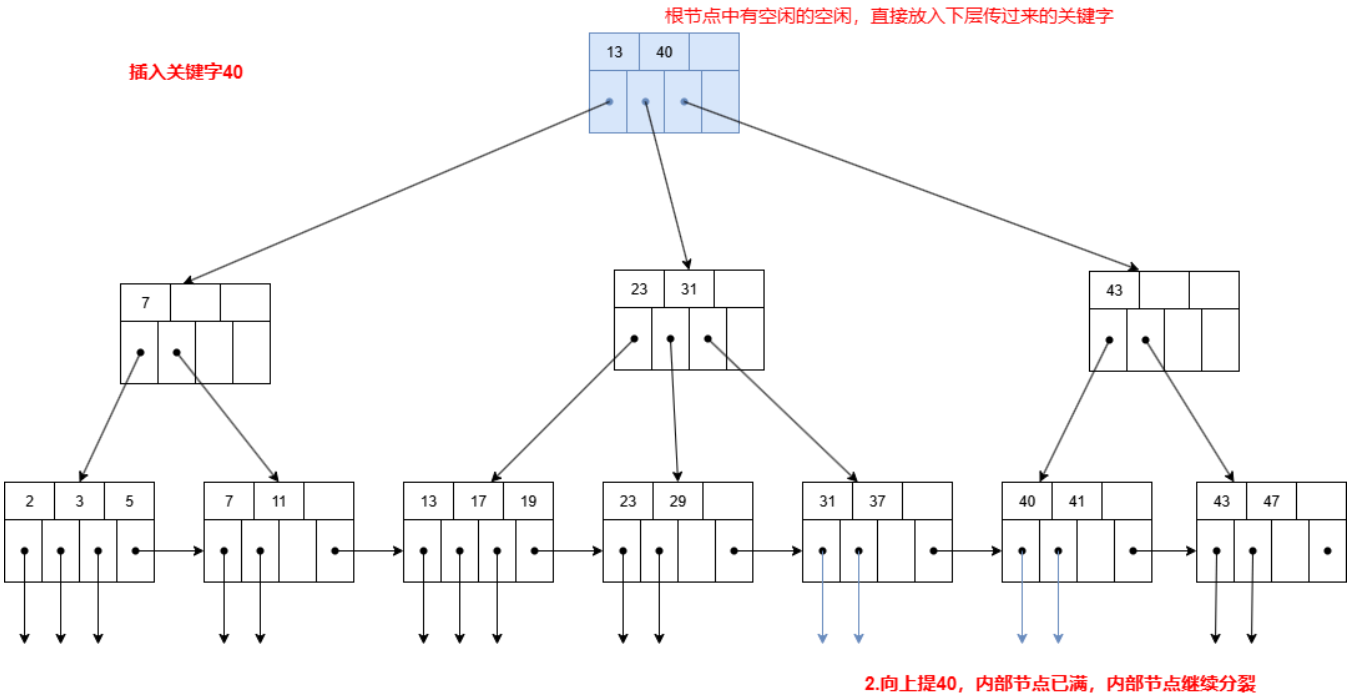
① 插入关键字40，叶子节点已满，分裂叶子节点



② 分裂后向上提中间值，发现内部节点已满，继续分裂内部节点



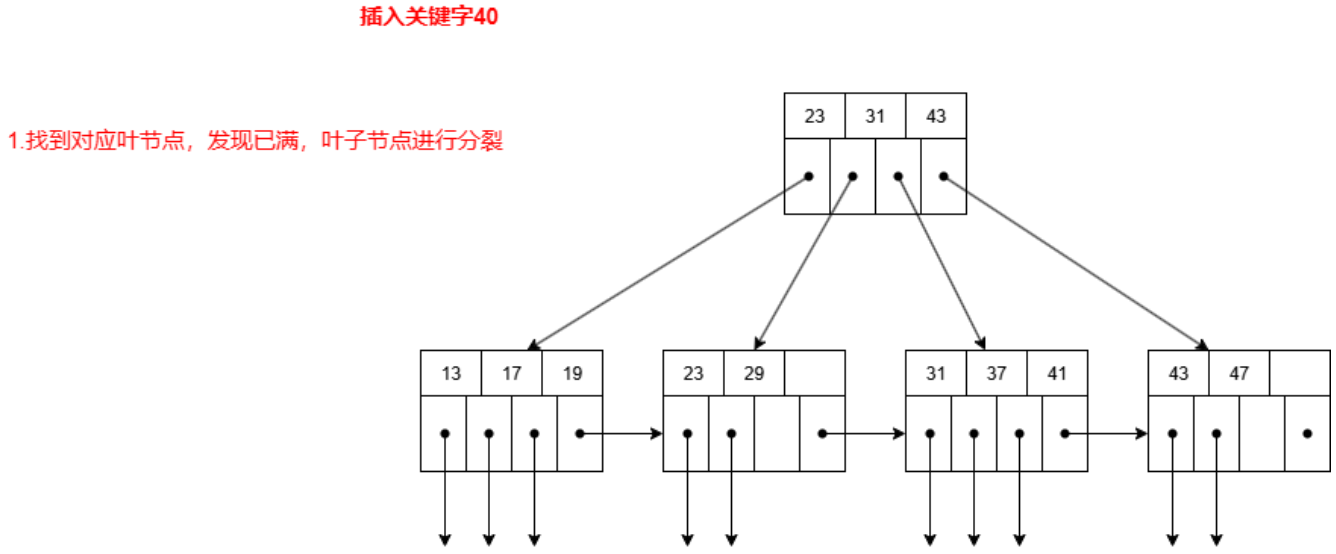
③ 分裂内部节点后，将内部节点的中间值提上去



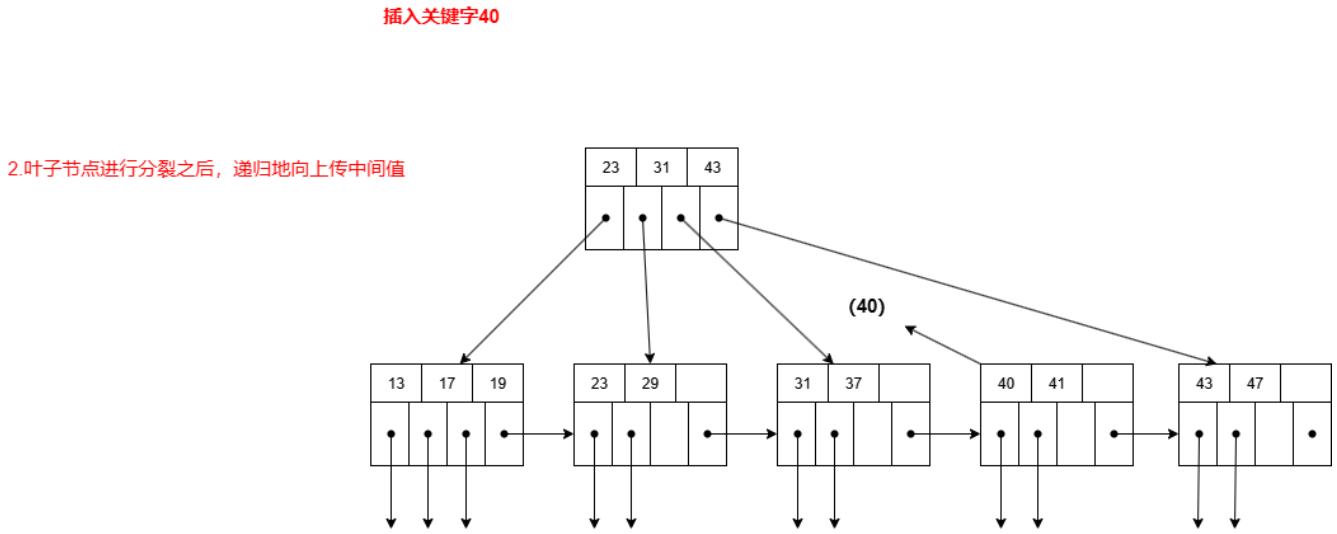
如果内部节点满了之后，就需要继续向父节点上传中间值，在本图中只有三层，所以内部节点的父节点即是根节点，如果根节点还存在空闲的空间，就直接在根节点中合适的位置上插入下层传过来的值。如果根节点也已经满了，继续向下看。

2. **内部节点满，根节点无空闲空间**

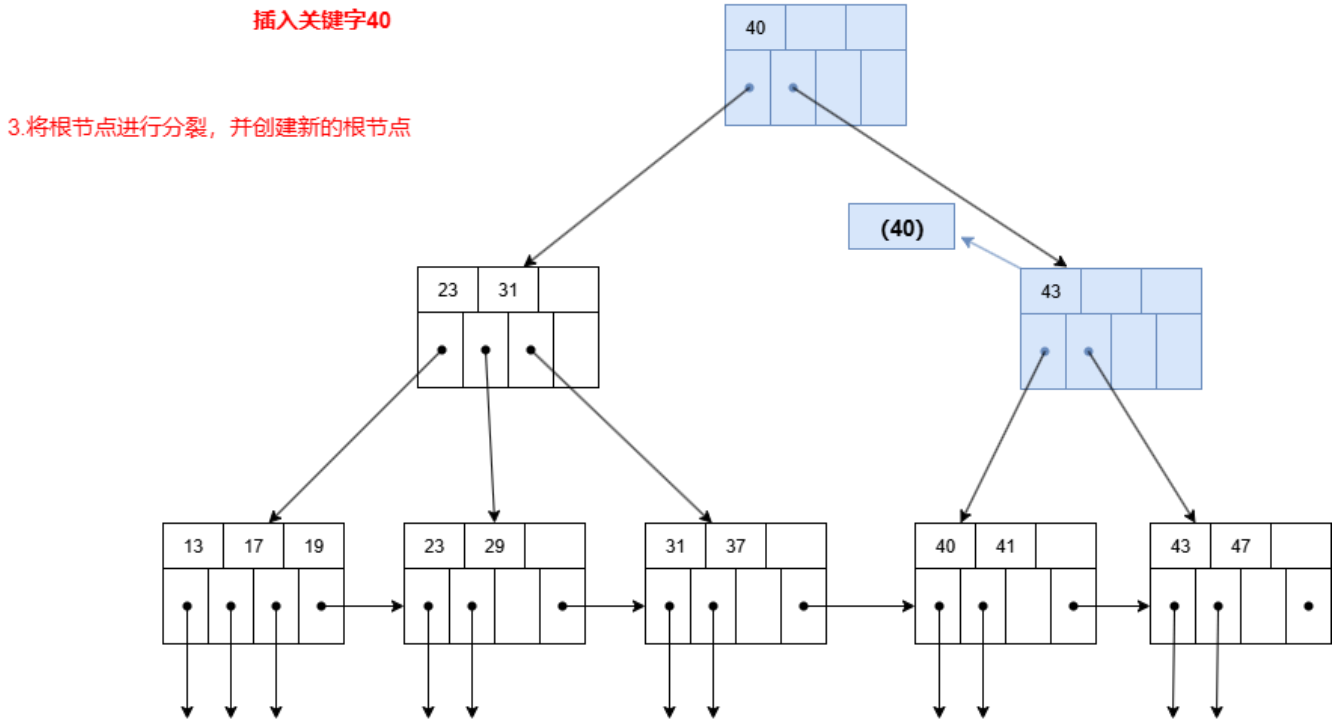
① 插入关键字40，发现节点已满



② 叶节点进行分裂，将中间值递归的提到上一层



③ 根节点已满，继续分裂，并且创建新的根节点



如果根节点也满了之后，就需要对根节点进行分裂了，对根节点进行分裂后，还需要创建一个节点作为新的根节点，新的根节点中存储下层传过来的唯一关键字。

B+树的删除

对于m阶的B+树，每个节点最多可以存储m个子节点，m - 1个关键字。删除一个元素的时候，首先找到该关键字所在的叶子结点，然后判断该关键字是否在该叶子节点中存在，若不存在，则此次删除操作执行结束。

概述：删除主要有以下的情况

（1）如果节点内的关键字删除之后满足半满的情况，直接退出

（2）如果节点的关键字删除之后不满足帮忙的情况，首先向兄弟借，如果能借，就向兄弟节点中借一个关键字，如果不能借的话，就要和兄弟节点进行合并了。

（3）合并会导致上层节点的键值减少，就需要递归的进行处理了，重复上述逻辑，直到满足B+树平衡条件，或者到B+树的根节点，此时删除后只有一个孩子，直 接将唯一的孩子作为新的根节点。

详情：

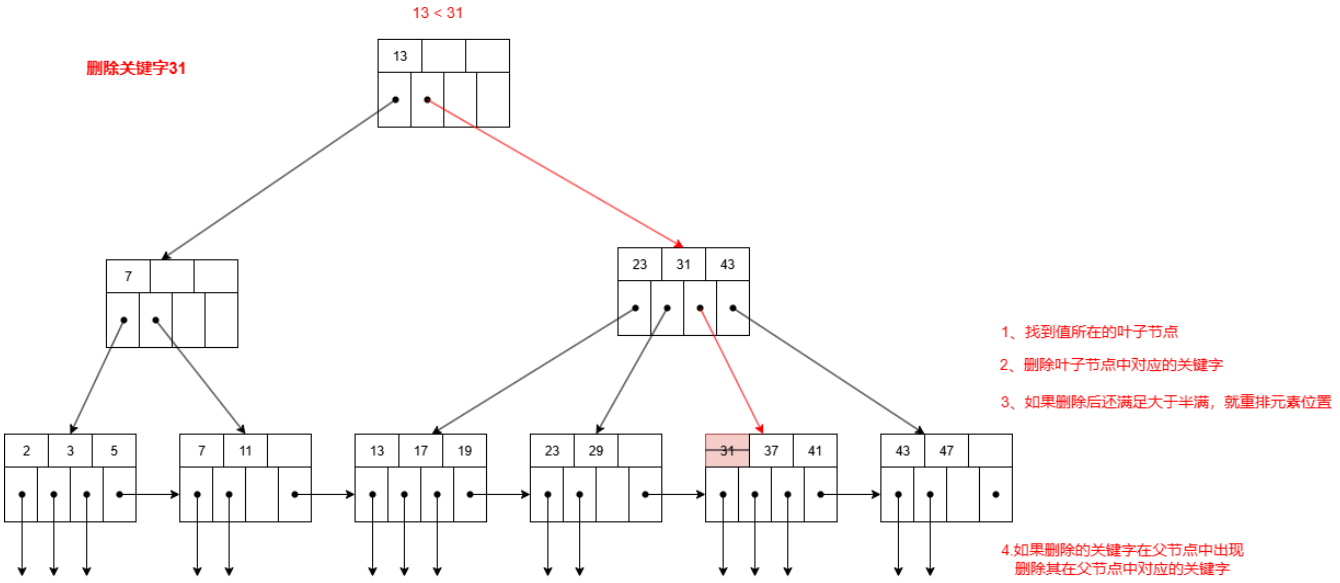
若该关键字在该叶子节点中存在，那么就删除该关键字以及对应的记录。

1. 若删除后该节点关键字的数量大于等于【m / 2 - 1】,说明结点满足平衡条件，此次删除操作逻辑结束。
2. 若删除后该节点的关键字的数量小于【m / 2 - 1】，说明该节点不满足平衡条件了，需要向兄弟节点借一个关键字。
    1. 如果兄弟节点（通常是左兄弟）能借出关键字，借之后结束此次删除逻辑。
    2. 如果兄弟节点也不能借出关键字，那么就要合并了
3. 与兄弟节点（通常是左兄弟）进行合并（将两个叶子节点及数据合并为一个）后，如果删除的节点的最小的关键字在父节点中出现的话，就需要删除在父节点中原两节点的分界点，同时父亲节点递归调整。
    1. 如果删除父节点中对应的关键字之后，父节点中的关键字的数量大于等于【(m - 1) / 2 - 1】,则满足平衡条件，退出此次删除逻辑。
    2. 如果删除父节点中对应的关键字之后，父节点中的关键字的数量小于【(m - 1) / 2 - 1】，那么父亲节点也要向其兄弟节点尝试借的操作。
        1. 如果该父亲节点的兄弟节点能借的话，就将兄弟节点中的最值拿过来。
        2. 如果该父亲节点的兄弟节点也不能借的话，继续合并内部节点，并将根节点中对应的关键字下移。
    3. 递归操作父亲节点的父亲节点。
4. 如果一直递归到根节点，且根节点只有一个子节点,就删除根节点，整棵树高度减1.

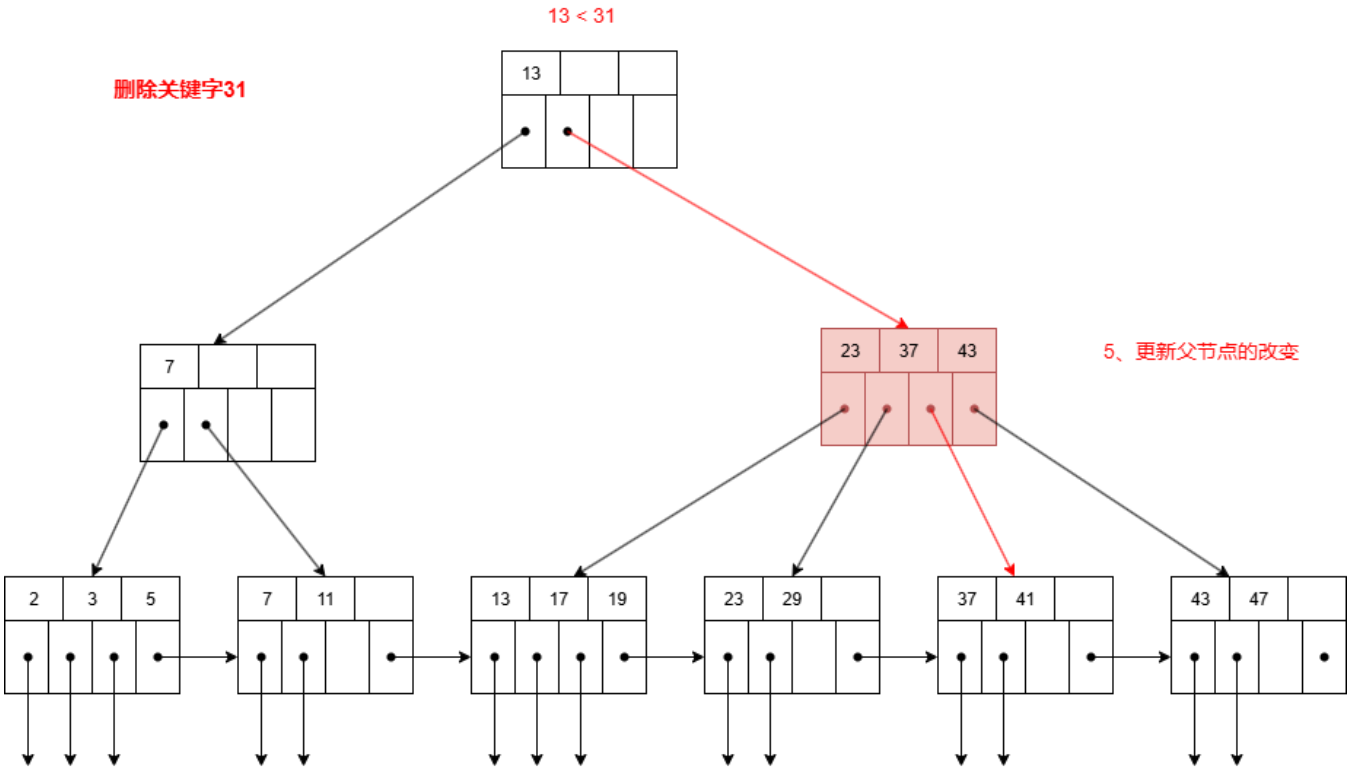
示例：

（1）叶节点满足删除之后大于等于【(m - 1) / 2 - 1】

①删除关键字及对应的记录

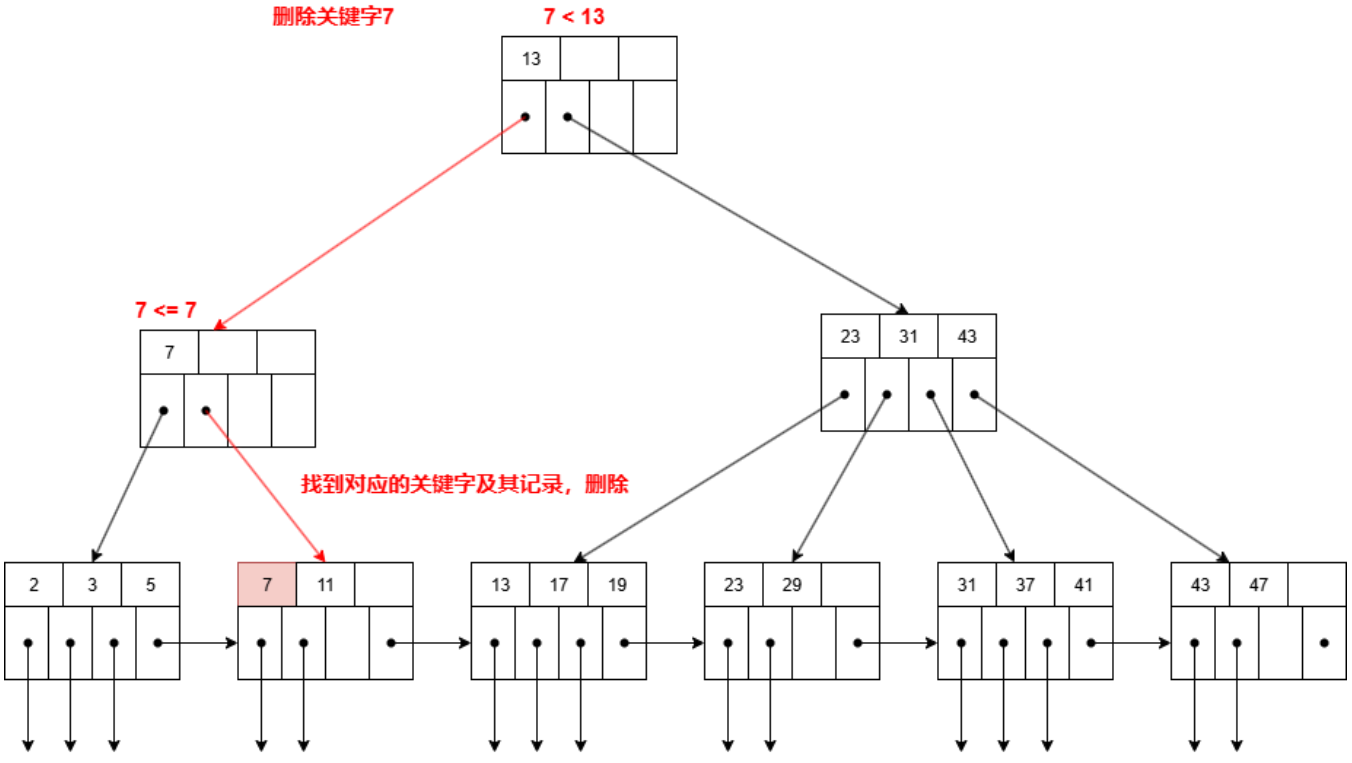


② 如果父节点中存在删除的关键字，对父节点进行改变

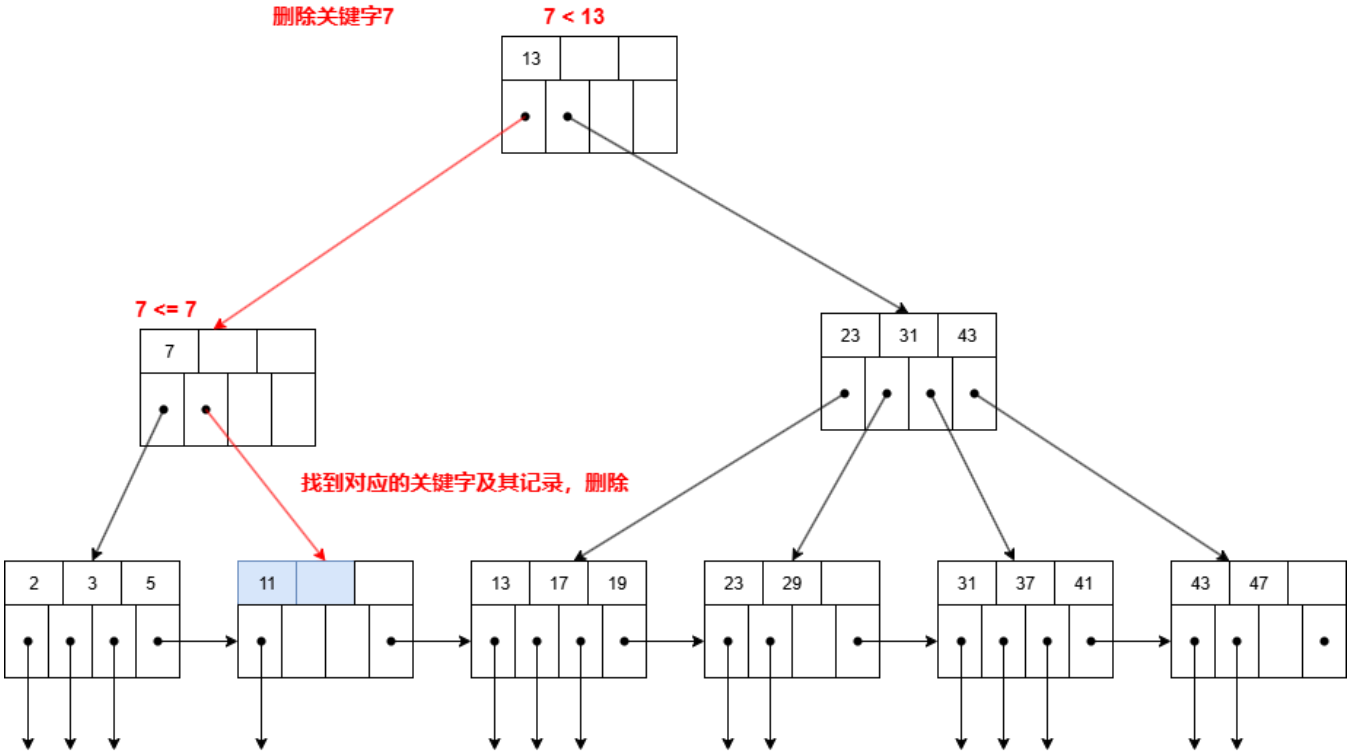


（2）删除关键字之后，叶节点关键字数量小于【(m - 1) / 2 - 1】，且可以向兄弟节点借

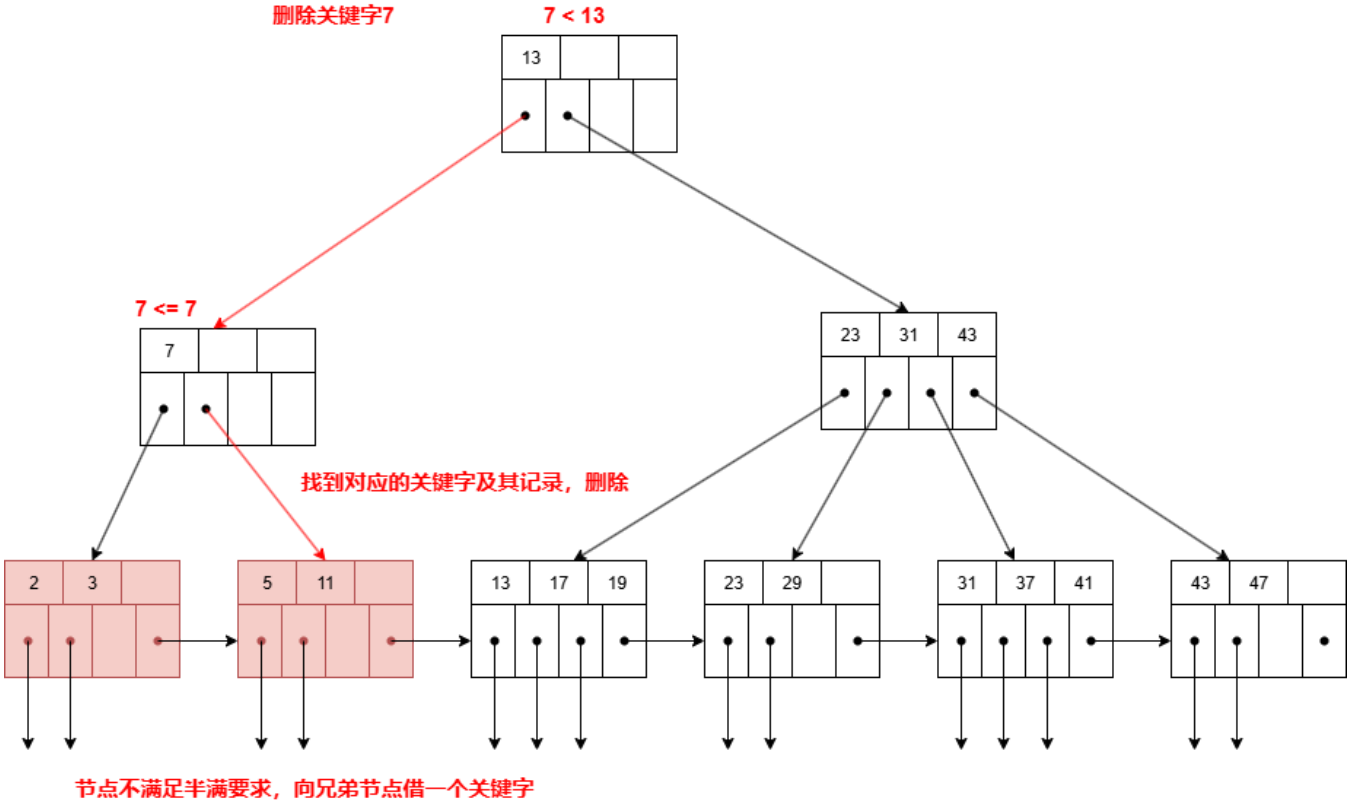
① 在B+树中找打对应的叶子结点



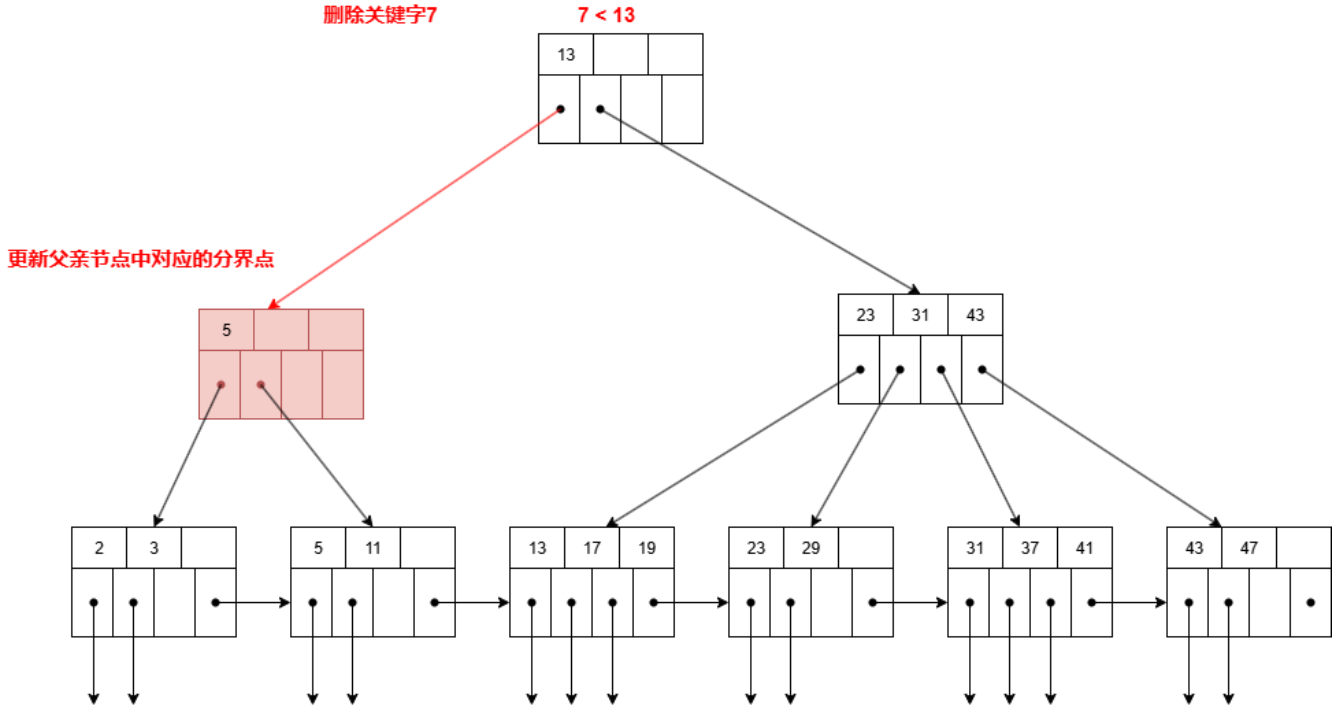
② 在叶子节点中删除关键字



③ 删除后，不满足半满的要求，向兄弟节点借一个



④ 修改父亲节点中对应的分界点

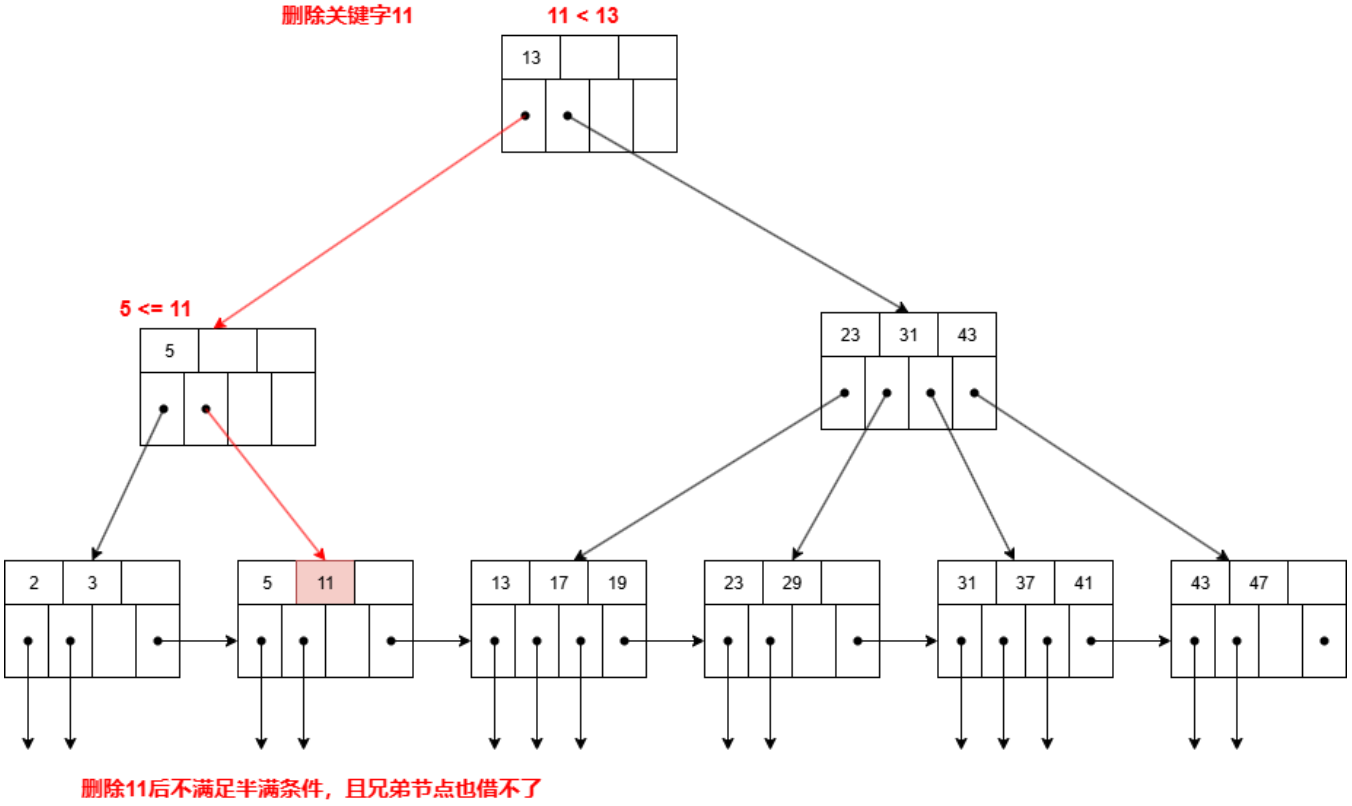


这里需要注意的是【5,11】和【13，17，19】不属于兄弟节点，他们的父亲不是同一个父亲

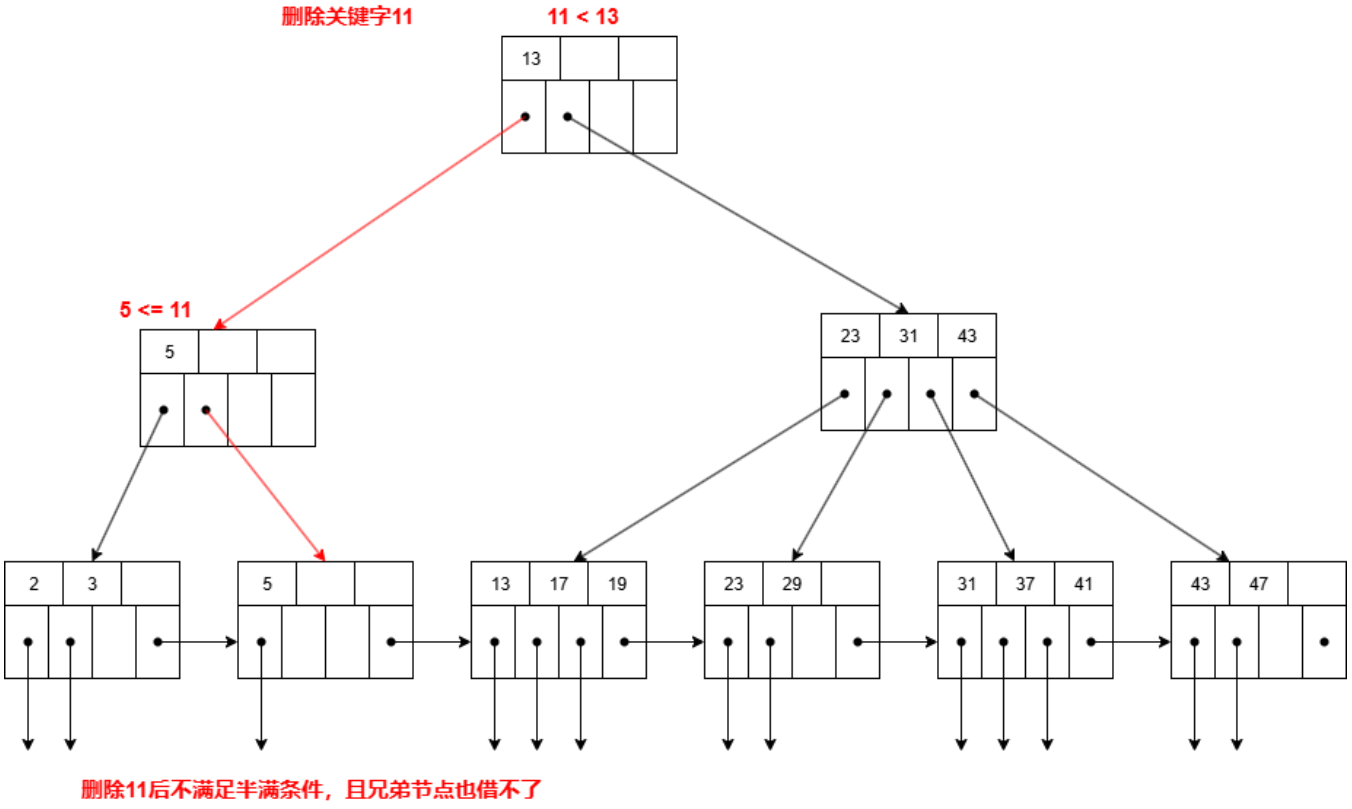
（3）删除关键字后不满足半满的条件，且兄弟节点也借不了

- 删除关键字11

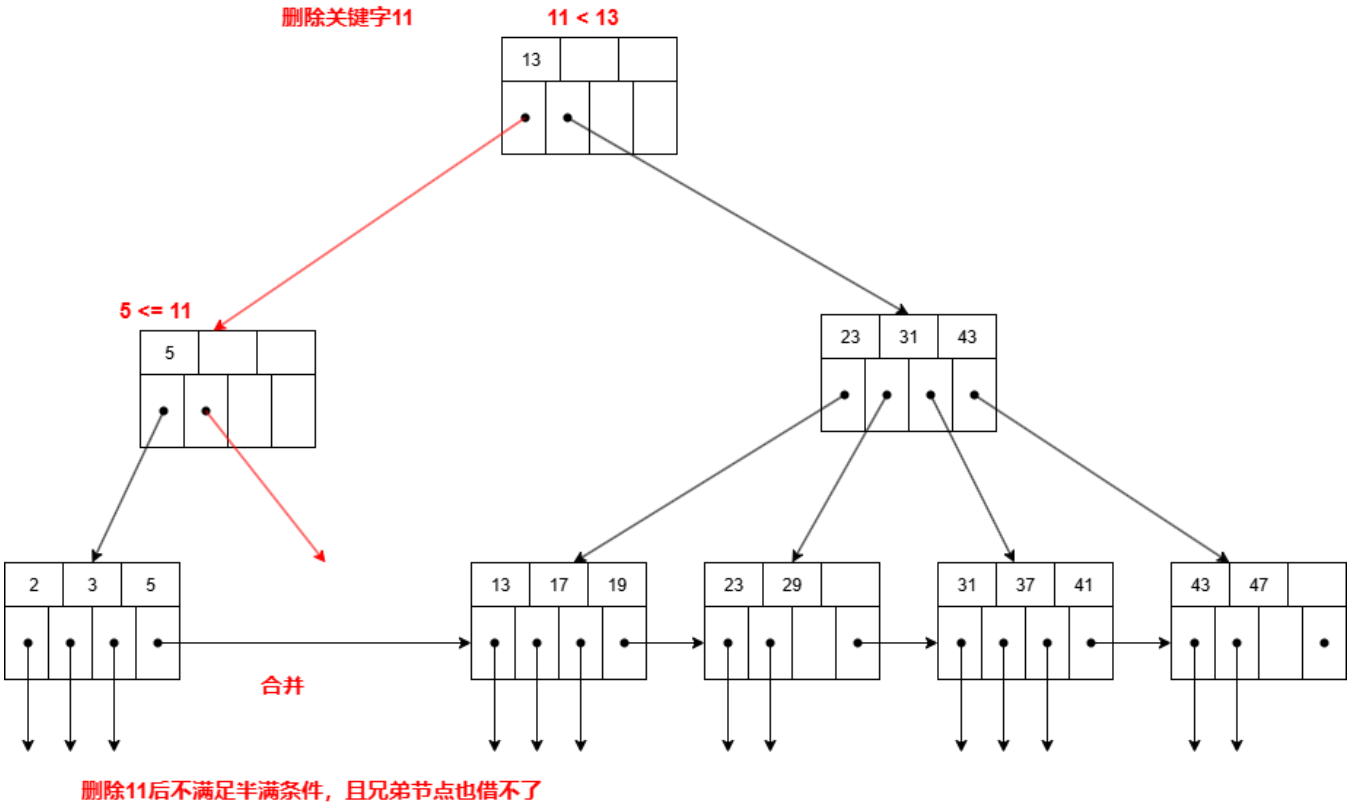
① 在B+树中找到对应的节点



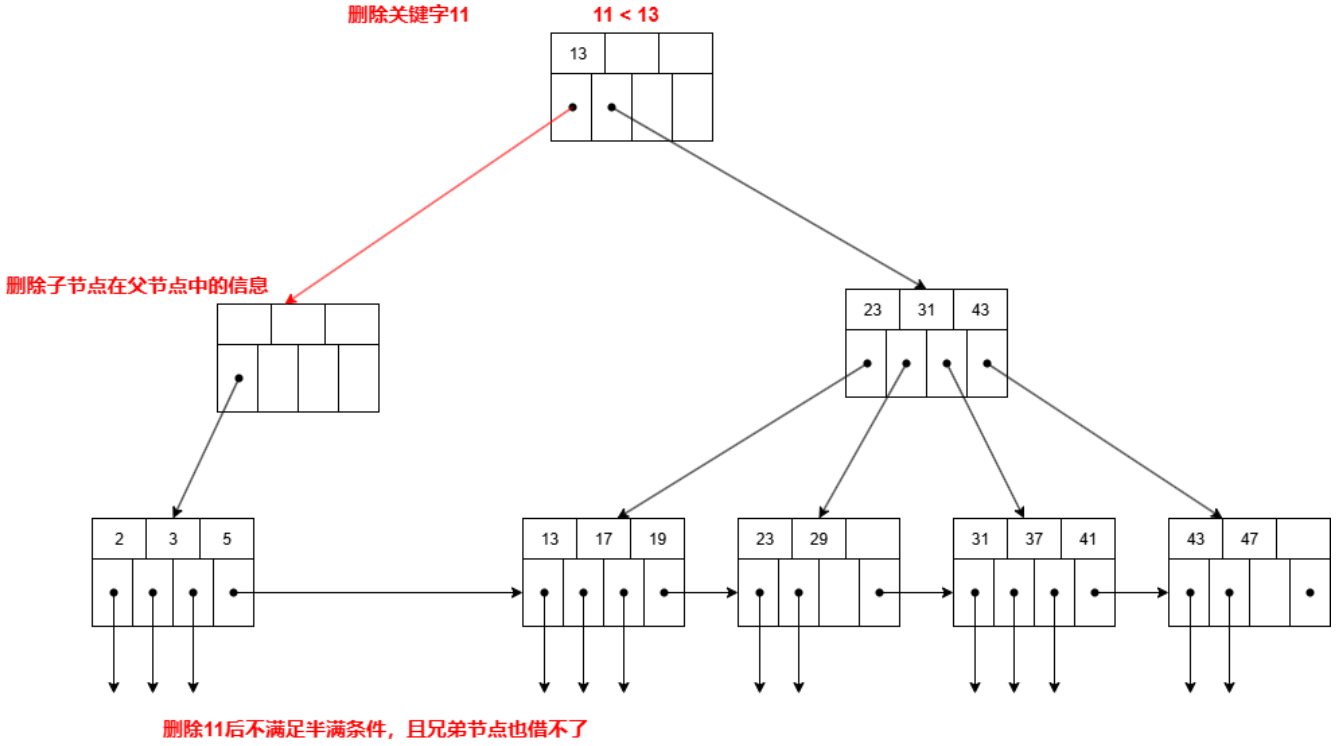
② 删除关键字



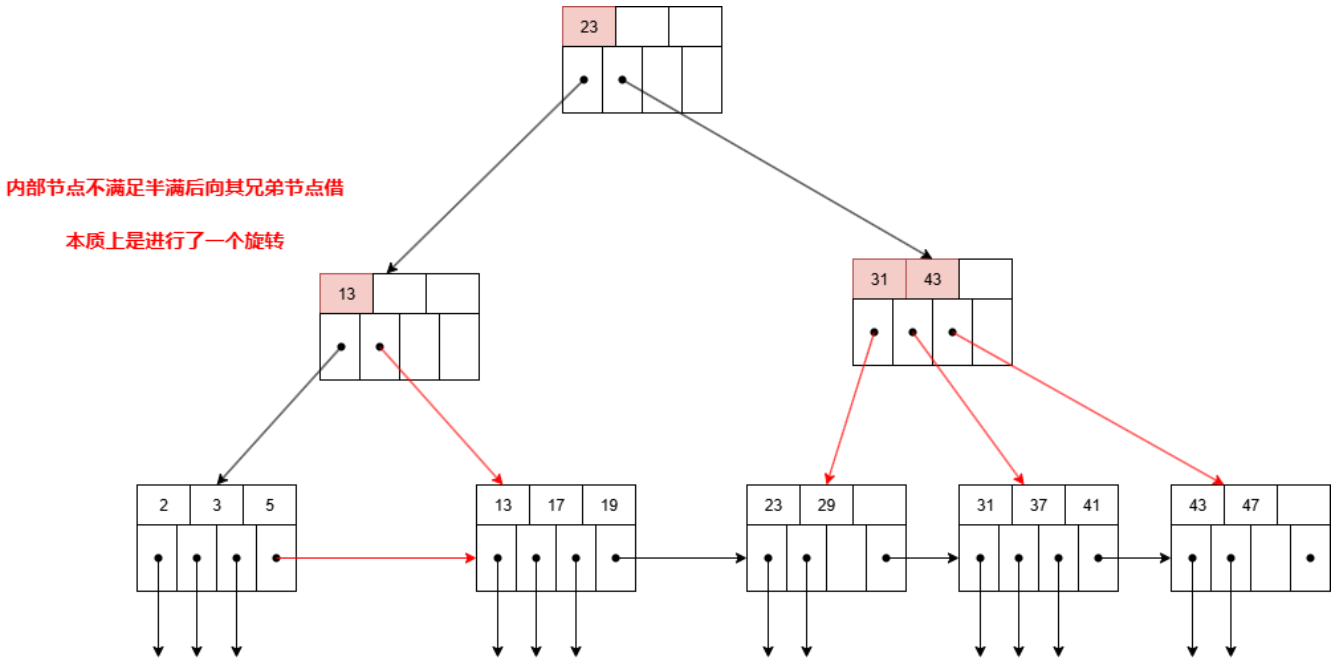
③ 不满足半满条件，并且兄弟节点也借不了，**和兄弟节点合并**



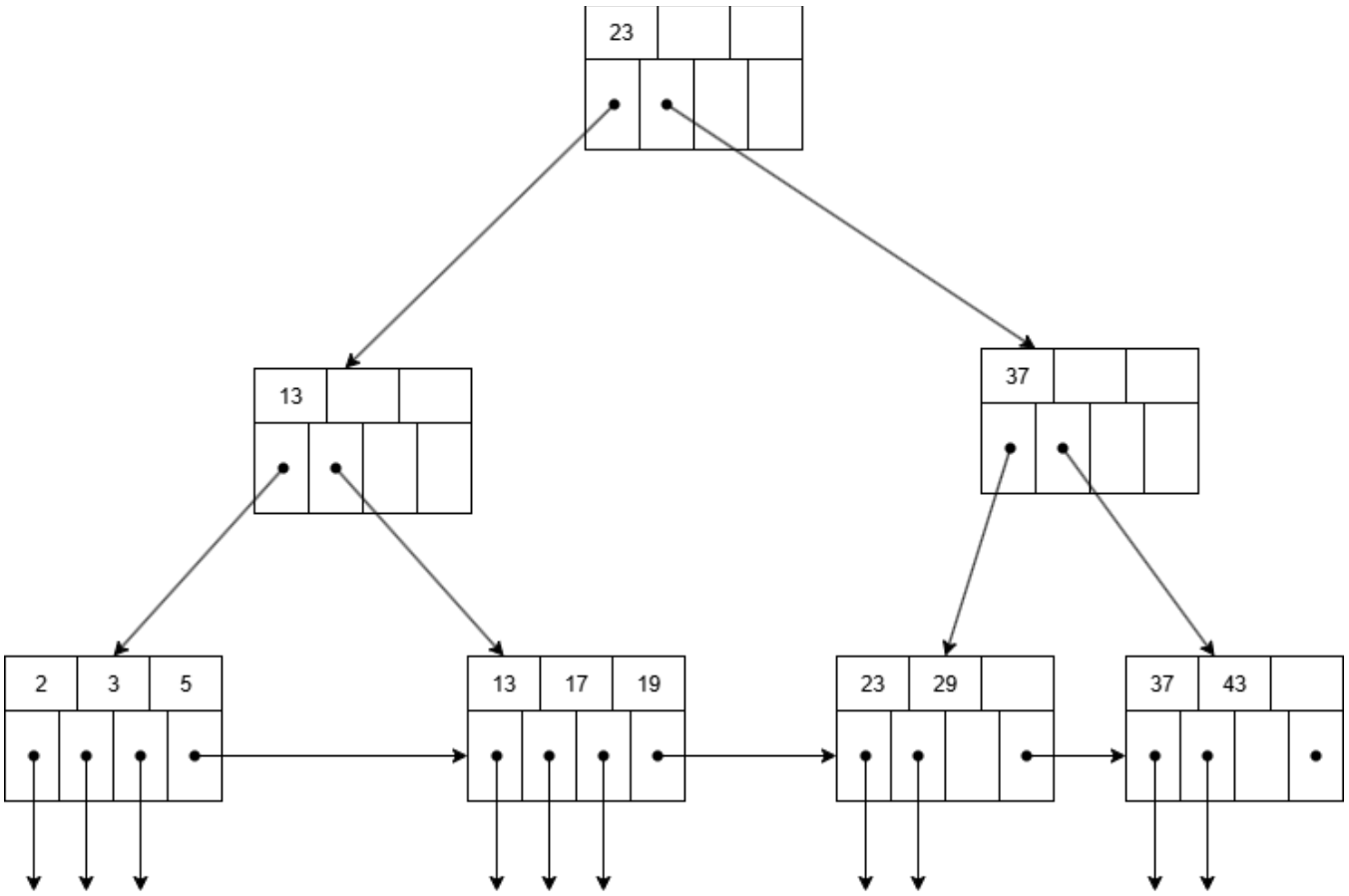
④ 删除子节点在父节点中的信息



⑤ 父节点（内部节点不满足半满条件）继续向其兄弟节点借

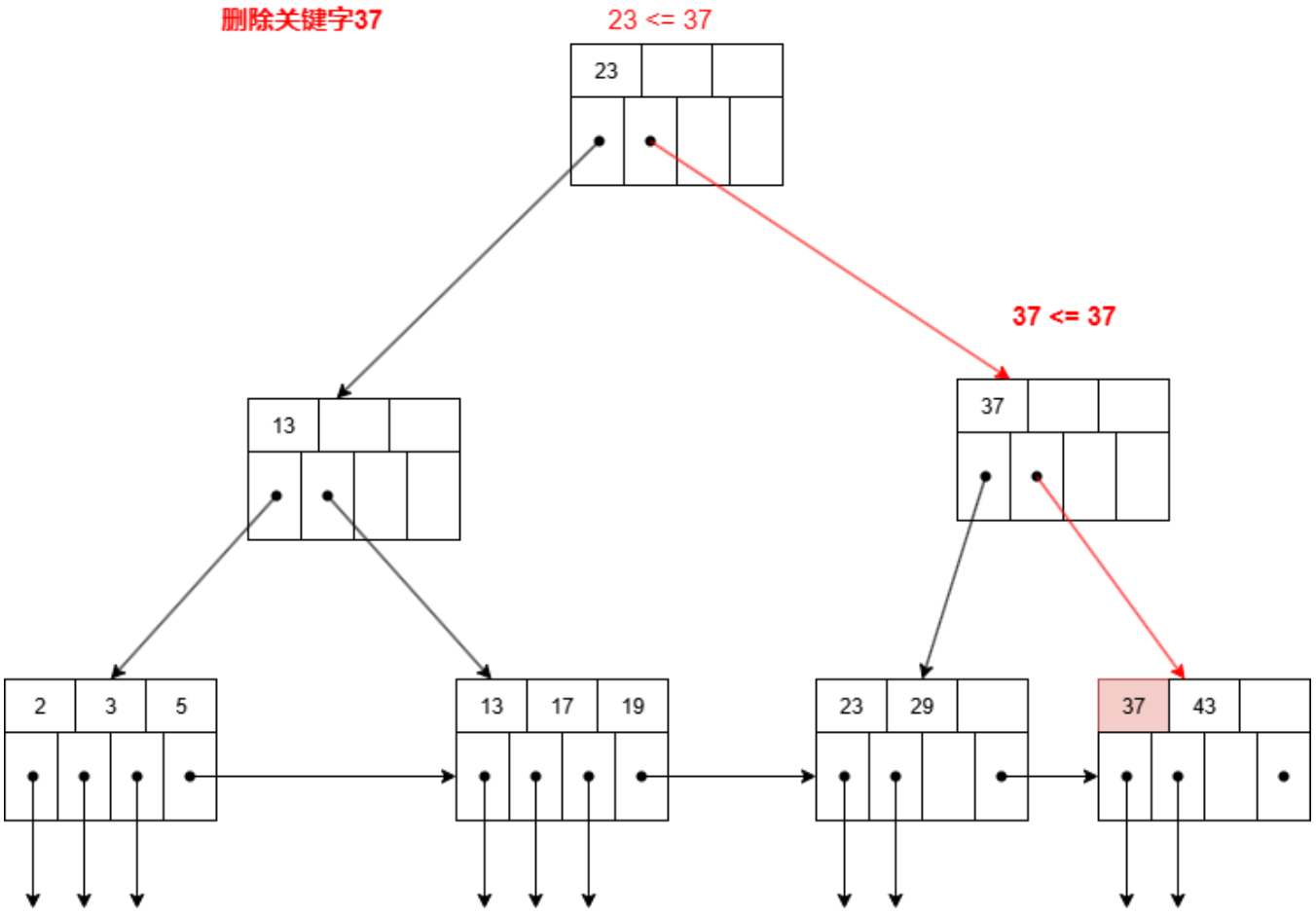


（4）删除叶节点之后，导致整体高度减1，根节点下移

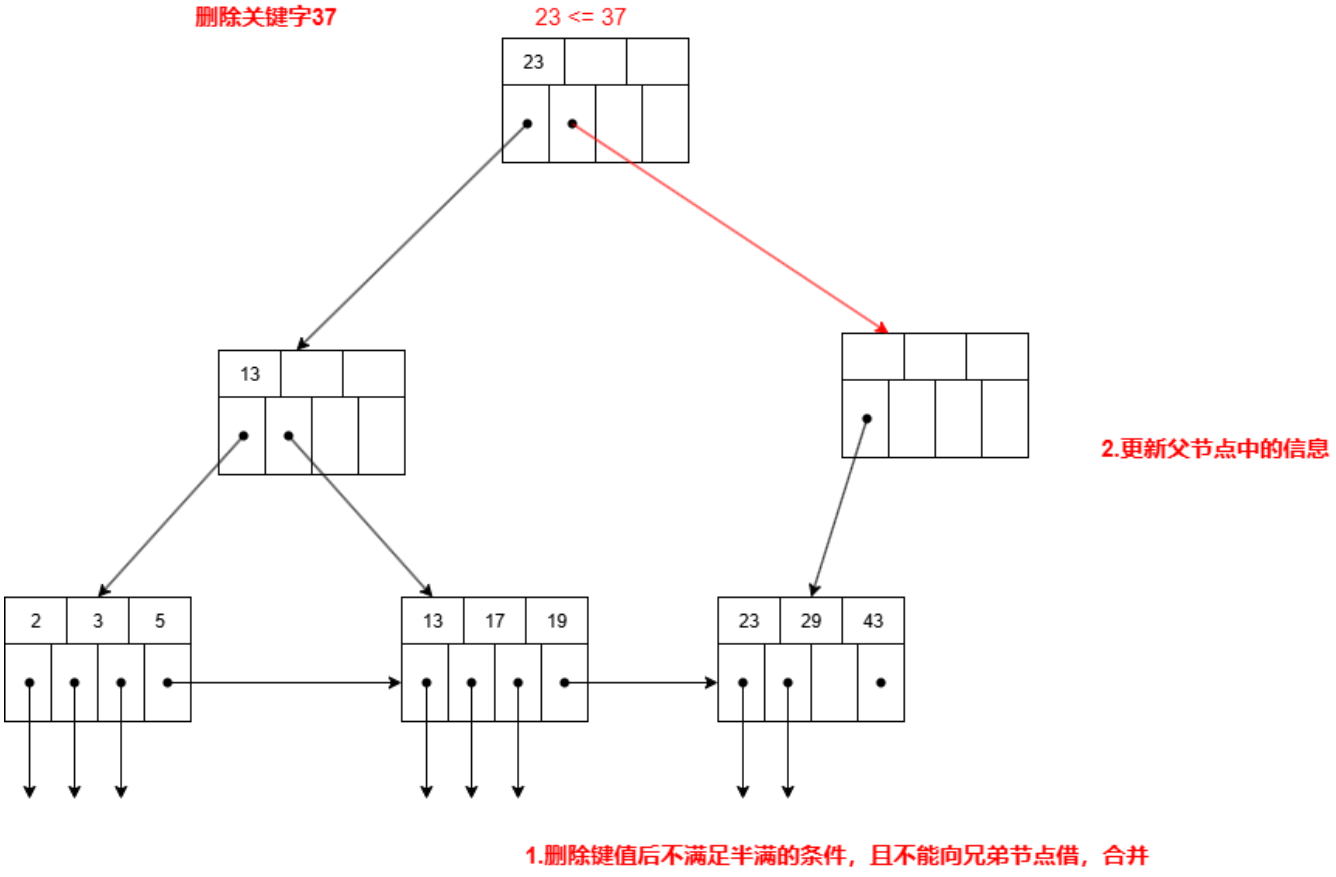


- 删除关键字37

① 找到关键字所在的叶子节点



② 删除关键字后不满足半满的条件，并且不能向兄弟节点借，合并



③ 内部节点不满足半满条件，且不能向兄弟节点借，将根节点下移

